

# MLA0303\_RL\_PROGRAM\_13.py

```
"""
Experiment 13: REINFORCE algorithm for an autonomous parking system to
learn optimal parking strategies. The car moves on a 1-D strip of
parking-lane cells and must stop exactly inside the marked parking bay.
"""

import numpy as np

np.random.seed(1)

LANE_LENGTH = 8
BAY = 5
ACTIONS = ["FORWARD", "BACKWARD", "STOP"]
GAMMA = 0.95
LR = 0.05
EPISODES = 3000

class ParkingEnv:
    def reset(self):
        self.pos = 0
        return self.pos

    def step(self, action):
        if action == 0:
            # FORWARD
            self.pos = min(LANE_LENGTH - 1, self.pos + 1)
            return self.pos, -1, False
        if action == 1:
            # BACKWARD
            self.pos = max(0, self.pos - 1)
            return self.pos, -1, False
        # STOP
        if self.pos == BAY:
            return self.pos, 25, True
        return self.pos, -10, True

theta = np.random.randn(LANE_LENGTH, len(ACTIONS)) * 0.01
ENTROPY_COEF = 0.02

def softmax(x):
    e = np.exp(x - np.max(x))
    return e / e.sum()

def choose_action(state):
    probs = softmax(theta[state])
    return np.random.choice(len(ACTIONS), p=probs), probs

def train():
    env = ParkingEnv()
    reward_history = []
    baseline = 0.0
    for ep in range(EPISODES):
        state = env.reset()
        trajectory = []
        for _ in range(20):
            action, probs = choose_action(state)
            next_state, reward, done = env.step(action)
            trajectory.append((state, action, reward, probs))
            state = next_state
            if done:
                break
        G, returns = 0, [0] * len(trajectory)
        for t in reversed(range(len(trajectory))):
            G = trajectory[t][2] + GAMMA * G
            returns[t] = G
        episode_return = sum(r for _, _, r, _ in trajectory)
        baseline = 0.95 * baseline + 0.05 * episode_return
        for (s, a, r, probs), G_t in zip(trajectory, returns):
            advantage = G_t - baseline
            grad = -probs
            grad[a] += 1
            entropy_grad = -probs * (np.log(probs + 1e-8) + 1) # encourages exploration
            theta[s] += LR * (advantage * grad + ENTROPY_COEF * entropy_grad)
        reward_history.append(episode_return)
    return env, reward_history

if __name__ == "__main__":
    env, rewards = train()
    print(f"Average return (first 50 episodes): {np.mean(rewards[:50]):.2f}")
    print(f"Average return (last 50 episodes) : {np.mean(rewards[-50:]):.2f}")

    state = env.reset()
    path = [state]
    for _ in range(15):
        action = int(np.argmax(theta[state]))
        state, reward, done = env.step(action)
        path.append(state)
        if done:
            break
    print("Learned parking manoeuvre (positions):", path)
    print("Final action outcome reward:", reward)
```

## PROGRAM OUTPUT

Average return (first 50 episodes): -6.28  
Average return (last 50 episodes) : 20.00  
Learned parking manoeuvre (positions): [0, 1, 2, 3, 4, 5, 5]  
Final action outcome reward: 25